THE RACK MONITOR APP

Contents

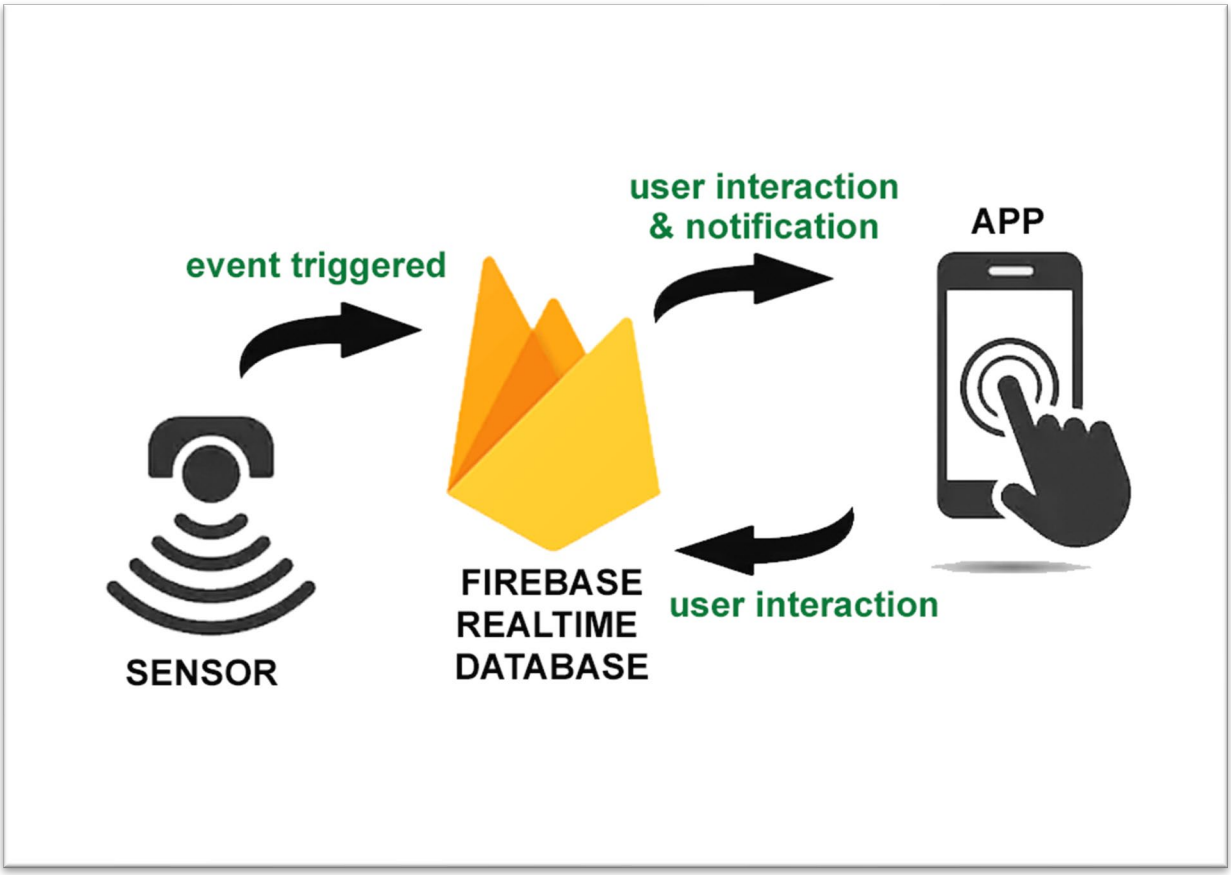
THE RACK MONITOR APP 1
Brief description of app 2
Summary 2
Back end database 3
Database schema 3
USER (admin generated data) 3
COMPANY (admin generated data) 4
SITE (admin or user generated data) 4
RACK BLOCK (admin or user generated data) 4
SENSORS (module generated data) 4
LOCATION (admin or user generated data) 4
INCIDENT 5
ini files 5
The Arduino hardware module 7
PCB design 7
Hardware functionality for Arduino sketch 9
Power up / battery change 10
Vibration detection 12
Battery monitoring 14
Appendix A 16

Brief description of app

The app will be a real time snapshot of the health of a rack post that has a shock sensor attached to it. The sensor will send an event to the firebase database which will then trigger a notification to the user's app that a shock event has occurred.
The user can then interact by manually with the database by inspecting the impacted item and feeding back accordingly as to the severity of the impact and call to action.
The sensor will interact natively with the real time db while the user will interact via the app. The app and sensor will not interact in any way with each other. This document focuses on the design and build of the database structure and the Arduino module.

Summary

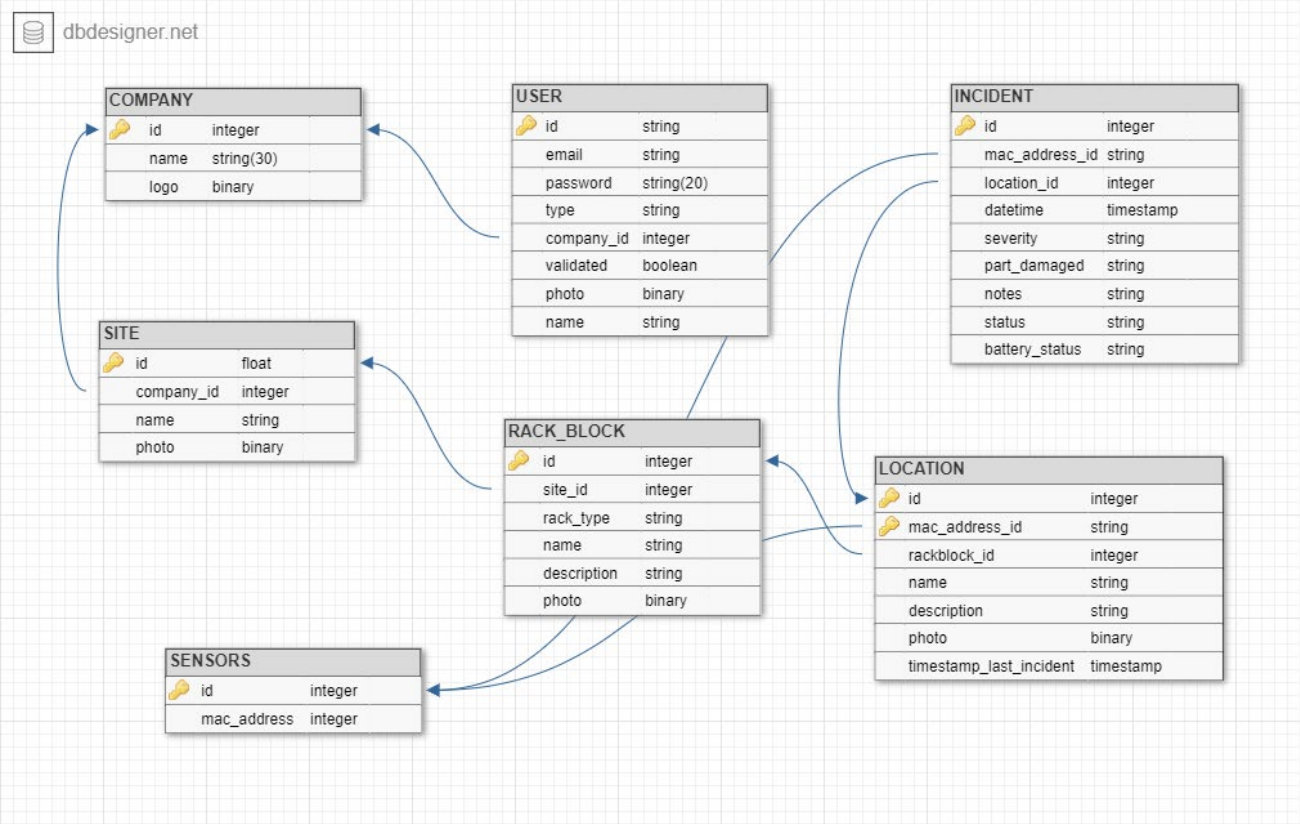
Below is a summary diagram of how the app will work



Back end database

Database schema

The database schema is depicted in the figure below:



The SQL schema for the above is shown in appendix a for creation in MySQL database (for informational purposes only)

USER (admin generated data)

- name (string)
- email/username (email) –primary key
- password (alphanumeric password)
- type (string)
    - user
    - admin
- company_id (numeric) – many users allowed per company
- validated (Boolean) – first time is set to false when setup by admin or if user asks to reset password
- photo (image)

COMPANY (admin generated data)

- company_id (Int) –primary key
- name (string)
- logo (image)

SITE (admin or user generated data)

- id (int) – primary key
- company_id / foreign key (int) – many sites allowed per company
- name (string)
- photo (image) –optional

RACK BLOCK (admin or user generated data)

- id / system generated, (int) –primary key
- site_id / foreign key / system generated, (int) – many rack blocks allowed per site
- rack type
- name / user generated, (string)
- description / user generated, (string)
- photo / user selection (image)

SENSORS (module generated data)

- id / system generated, (int) –primary key
- mac_address / module generated, (string)

LOCATION (admin or user generated data)

- id / system generated, (int) –primary key
- mac_address_id / admin generated, (string) –primary key / foreign key
- rackblock_id as foreign key / system generated, (int) – many locations allowed rack block
- name / user generated, (String)
- mac_address / admin generated (string)
- battery_status / sensor generated, (String)
- status / user or admin generated (string)
- timestamp_last_incident / system generated (timestamp)

INCIDENT

- id / system generated, (int) –primary key
- mac_address_id / admin generated – foreign key
- location_id as foreign key / system generated, (int) – many incidents allowed per location
- date / system generated, (timestamp)
- severity / user generated, (string) – std values from an ini file
- part_damaged / user generated, (array) – multiple parts allowed – see ini file section
    - post
    - bracing
    - foot plate
    - fixing
    - protector
    - beam
    - pin
- notes / user generated, (string)
- status / user generated, (String) – see ini file section
    - new impact
    - identified and called for fix
    - fixed and closed
- battery status

ini files

STATUS

- normal
- new impact
- identified and called for fix
- fixed and closed

PART DAMAGED

- post
- bracing
- foot plate
- fixing
- protector

- beam
- pin

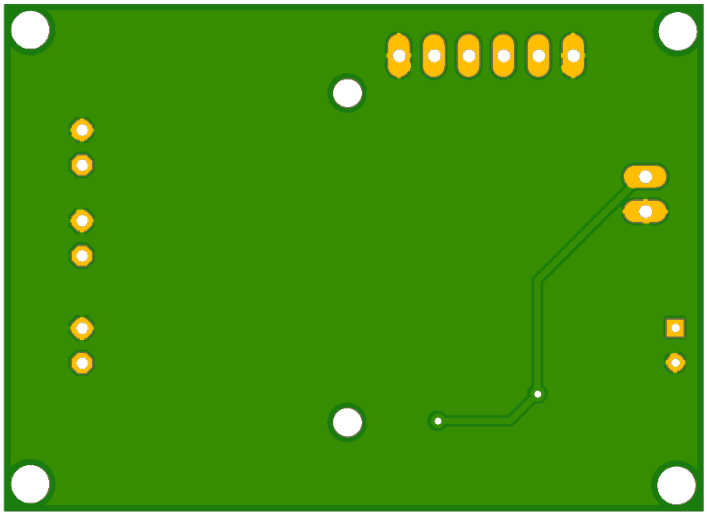
SEVERITY

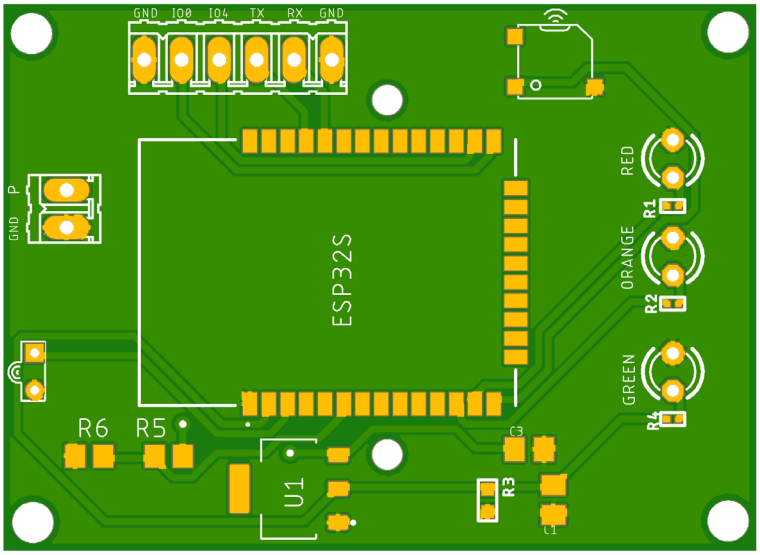
- RED
- AMBER
- GREEN

The Arduino hardware module

PCB design

The hardware will be based on an Arduino ESP 32S chip and constructed using the design and BOM file as shown below (all files can be found at (https://spima.box.com/s/cot4p94ns9aceho78ka9c67eq890d6yb):




GND IO0 IO4 TX RX GND
GND P
ESP32S
RED
R1
ORANGE
R2
GREEN
R4
R6 R5
U1
C3
R3
C1

Part	Value	Device	Package	Description
BATTERY	MP-4-3	M02	02P	AMP QUICK CONNECTOR
C1	1000uF	C-USC0805K	C0805K	CAPACITOR, American symbol
C3	100uF	C-USC0805K	C0805K	CAPACITOR, American symbol
ESP32S	ESP-32S	ESP-32S	ESP-32S	WiFi-BT-BLE MCU module
GREEN	Green	LED3MM	LED3MM	LED
ORANGE	Orange	LED3MM	LED3MM	LED
R1	ERJ-UP3J221V	R_0201	R0201	RESISTOR
R2	ERJ-UP3J221V	R_0201	R0201	RESISTOR
R3	RNCF0603DTE10K0	R_0603	R0603	RESISTOR
R4	ERJ-UP3J221V	R_0201	R0201	RESISTOR
R5	RMCF0201JT10K0	R-US_R0805	R0805	RESISTOR, American symbol
R6	RMCF0201FT15K0	R-US_R0805	R0805	RESISTOR, American symbol
RED	RED	LED3MM	LED3MM	LED
SL1	Adafruit 2135	M06	06P	AMP QUICK CONNECTOR
SPK1	SMT-0540-S-R	SMT-0540-S-R	SMT-0540-S-R	SMT-0549-S-R
U$1	SW-18015P	SW-18015P	SW-18015P	
U1	AMS1117-3.3	AMS1117-3.3	SOT229P700X180-4N	Sot223/Pkg 1-Amp 3.3-Volt Low Drop Out Voltage Regulatator

Designator	Comment	Layer	Footprint	Center-X(mil)	Center-Y(mil)	Rotation
R6	RMCF0201FT15K0	TopLayer	R0201	299.21	590.55	180
R5	RMCF0201JT10K0	TopLayer	R0201	437.01	562.99	90
R3	RNCF0603DTE10K0	TopLayer	R0603	1307.09	433.07	270
U$1		TopLayer	SW-18015P	98.43	776.77	90
U1	AMS1117-3.3	TopLayer	SOT229P700X180-4N	776.38	463.78	90
SPK1		TopLayer	SMT-0540-S-R	1486.61	1602.76	270
SL1	Adafruit 2135	TopLayer	06P	640.94	1607.48	0
R4	ERJ-UP3J221V	TopLayer	R0201	1801.18	649.39	0
R2	ERJ-UP3J221V	TopLayer	R0201	1801.18	957.18	0
R1	ERJ-UP3J221V	TopLayer	R0201	1801.18	1217.72	0
BATTERY		TopLayer	02P	183.86	1210.24	90
RED	RED	TopLayer	LED3MM	1801.18	1343.92	90
ORANGE	Orange	TopLayer	LED3MM	1801.18	1083.37	90
GREEN	Green	TopLayer	LED3MM	1801.18	775.59	90
ESP32S	ESP-32S	TopLayer	ESP-32S	1027.75	1040.16	90
C3	100uF	TopLayer	C0805K	1418.11	568.9	0
C1	1000uF	TopLayer	C0805K	1482.28	434.25	270

Hardware functionality for Arduino sketch

The functionality of the module consists of 2 parts; the battery monitoring and the detection of a vibration from the sensor.

Three LEDs will be connected to the battery pack that will monitor battery status on a 24 hr basis. The ESP will be woken up on the events outlined in the forthcoming description and will send data to the firestore, as well as activate a buzzer that is connected to. In addition to the buzzer a blue LED will indicate that the ESP has been woken up.

When the module gets powered for the first time it will send a battery status update to the firestore by adding an incident row to the incident table.
Further more the sketch will also be programmed to light the following LED's depending on battery voltage:

- > 1.3v, battery good, green LED lights
- >= 1.1v & <1.3, battery medium, orange LED lights
- <1.1v, battery weak, red LED lights

In the interest of prolonging the battery life of the module thereafter, it is necessary to operate the module the majority of the time in deep sleep mode and allow the module to wake up at specific events.
When woken up, the chip will send certain data to the firebase firestore. The events that trigger the wake up are the following:

- First time power up or battery change
- Vibration detection
- Battery monitoring

Below both events are described in addition to module power up.

Power up / battery change

This occurs when new batteries are inserted for the first time or the Arduino sketch is uploaded to the module.

The appropriate battery LED lights up continuously based on the battery voltage. The ESP wakes and ESP LED lights up signifying that its about to connect to the WIFI and send data.

A new row is added to the sensor table to register the new device on the database.

The following example shows a typical data entry – highlighted cells shows new data entered. The mac address is unique hence if the batteries are replaced in the future it will not register in the table again. Before making an insert a query will be run to see if the mac address already exists first.

SENSOR TABLE	
Id	0001
Mac_address	987239238

If the device is powered for the first time, the user then sets up the locations in the rack block. They will select from a list of sensors already powered up and registered to create the location.
An incident row is also recorded within the incident table upon powerup and inputs the battery status and datetime into the table. Below is an example. The following example shows a typical scenario – highlighted cells shows new data entered. The mac_address_id must be queried from the sensor table first.

SELECT SENSOR.id from SENSOR where mac_address = 987239238
INSERT INTO INCIDENT TABLE AS BELOW using mac_address_id = value above

INCIDENT TABLE	
Id	0001
Location_id	null
Mac_address_id	0001
Timestamp	1569853522
Severity	Null
Part_damaged	Null
notes	Null
status	Null
Battery_status	Good

The firestore then runs a query with the sensor id from above, in order to retrieve the location record and perform an update – must be part of a complete commit/rollback function.

SELECT LOCATION.id FROM LOCATION WHERE mac_address_id = value above
UPDATE LOCATION TABLE USING VALUES BELOW WHERE id = value above

LOCATION TABLE	
Location_id	0001
Rackblock_id	0001
Name	A1
Mac_address	5626562
Battery_status	Good
Status	New Impact
Trimestamp_last_incident	1569853522

For the first time power up, a location record will not exist (until the admin sets one up) hence a location update cannot be performed. So if a record fails to be found in the location table (only for first time power up), the whole function commits after the incident is recorded.

In the eventuality of a battery replacement, the location table will be updated as well.

After completion the ESP goes to sleep mode and awaits any of the events below.

Vibration detection

When a vibration is detected, the chip will awoken.

The appropriate battery LED lights up continuously based on the battery voltage. The ESP wakes and ESP LED lights up signifying that its about to connect to the WIFI and send data. The buzzer also sounds for 10s to signal a shock.

The following data sent to the firestore incident table:

- Message string “New impact”
- Location
- Date/time

The mac_address_id must be queried from the sensor table first.
SELECT SENSOR.id from SENSOR where mac_address = 987239238

INSERT INTO INCIDENT TABLE AS BELOW using mac_address_id = value above

The following example shows a typical scenario – highlighted cells shows new data entered

INCIDENT TABLE	
Id	0001
Mac_address_id	987239238
Timestamp	1569853522
Severity	Null
Part_damaged	Null

notes	Null
status	New impact
Battery_status	Null

The firestore runs a query using the new id created of the above incident to retrieve the location and perform an update on the record – must be part of a complete commit/rollback function.

LOCATION TABLE	
Location_id	0001
Rackblock_id	0001
Name	A1
Mac_address	5626562
Battery_status	Low
Status	New Impact
Trimestamp_last_incident	1569853522

After a successful commit, a push notification is sent to the user's device for an alert. By clicking the alert, the user is taken to the audit trail page for the specific mac address/location.

Additionally an e-mail is sent to the user. The text will be as follows:

Notification:
New impact detected at location {LOCATION.name}, on {INCIDENT.datetime}

e-mail to {USER.email}:
Dear {USER.name}

A new impact was detected at location {LOCATION.name}, on {INCIDENT.datetime}. Please take necessary steps to inspect the location and fix the damage.

Following the above procedure the chip goes back into deep sleep until 1 of the two events mentioned above trigger a wake up again.

Battery monitoring

The chip will wake up every 24 hrs @ 06:00 to measure the voltage across the power pack and send an entry into the incident table of the firestore.

The power pack will consist of 3XAAA batteries. The data sent will be the battery status and a timestamp of the time of reading.

The appropriate battery LED lights up continuously based on the battery voltage. The ESP wakes and ESP LED lights up signifying that its about to connect to the WIFI and send data. The buzzer does not sound on this occasion.

The voltage readings will also send an analog output to the relevant analog pins (LED lamps) as per the below occasions (voltages outlined are for per 1 battery):

- > 1.3v, battery good, green LED lights
- >= 1.1v & <1.3, battery medium, orange LED lights
- <1.1v, battery weak, red LED lights

The mac_address_id must be queried from the sensor table first.
SELECT SENSOR.id from SENSOR where mac_address = 987239238
INSERT INTO INCIDENT TABLE AS BELOW using mac_address_id = value above
An example data is shown below (the highlighted data shows the new ones, the others are copied from the previous):

INCIDENT TABLE	
Id	0001
Mac_address_id	987239238
Timestamp	1569853522
Severity	Null
Part_damaged	Null
notes	Null
status	Null
Battery_status	low

The firestore runs a query using the new id created of the above incident to retrieve the location and perform an update on the record – must be part of a complete commit/rollback function.

LOCATION TABLE	
Location_id	0001
Rackblock_id	0001
Name	A1
Mac_address	5626562
Battery_status	low
Status	Normal
Trimestamp_last_incident	1569853522

After a successful commit (and only if the battery status is beneath 1.3v), a push notification will be sent to the user's device for an alert. By clicking the alert, the user is taken to the audit trail page for the specific mac address/location.
Additionally an e-mail is sent to the user. The text will be as follows:

Notification:
{INCIDENT.battery_status} at location {LOCATION.name}, on {INCIDENT.datetime}

e-mail to {USER.email}:
Dear {USER.name}

The current battery status at {LOCATION.name} is {LOCATION.battery_status}. Please take steps to replace the battery in good time to avoid disruption of monitoring.

Following the above procedure the chip goes back into deep sleep until 1 of the two events mentioned above trigger a wake up again.

Appendix A

Database schema

```
CREATE TABLE `COMPANY` (
      `id` INT NOT NULL AUTO_INCREMENT,
      `name` VARCHAR(30) NOT NULL AUTO_INCREMENT,
      `logo` BINARY NOT NULL AUTO_INCREMENT,
      PRIMARY KEY (`id`)
);

CREATE TABLE `USER` (
      `id` VARCHAR(255) NOT NULL AUTO_INCREMENT,
      `email` VARCHAR(255) NOT NULL UNIQUE,
      `password` VARCHAR(20) NOT NULL UNIQUE,
      `type` VARCHAR(255) NOT NULL UNIQUE,
      `company_id` INT NOT NULL UNIQUE,
      `validated` BOOLEAN NOT NULL,
      `photo` BINARY NOT NULL,
      `name` VARCHAR(255) NOT NULL,
      PRIMARY KEY (`id`)
);

CREATE TABLE `INCIDENT` (
      `id` INT NOT NULL AUTO_INCREMENT,
      `mac_address_id` VARCHAR(255) NOT NULL UNIQUE,
      `location_id` INT NOT NULL,
      `datetime` TIMESTAMP NOT NULL,
      `severity` VARCHAR(255),
      `part_damaged` VARCHAR(255),
      `notes` VARCHAR(255),
      `status` VARCHAR(255) NOT NULL,
      `battery_status` VARCHAR(255) NOT NULL,
      PRIMARY KEY (`id`)
);

CREATE TABLE `SITE` (
      `id` FLOAT NOT NULL AUTO_INCREMENT,
      `company_id` INT NOT NULL,
      `name` VARCHAR(255) NOT NULL,
      `photo` BINARY,
      PRIMARY KEY (`id`)
);

CREATE TABLE `LOCATION` (
      `id` INT NOT NULL AUTO_INCREMENT,
      `mac_address_id` VARCHAR(255) NOT NULL UNIQUE,
      `rackblock_id` INT NOT NULL,
      `name` VARCHAR(255) NOT NULL,
      `description` VARCHAR(255),
      `photo` BINARY,
      `timestamp_last_incident` TIMESTAMP NOT NULL,
```

```
	PRIMARY KEY (`id`,`mac_address_id`)
);

CREATE TABLE `RACK_BLOCK` (
	`id` INT NOT NULL AUTO_INCREMENT,
	`site_id` INT NOT NULL,
	`rack_type` VARCHAR(255) NOT NULL,
	`name` VARCHAR(255) NOT NULL,
	`description` VARCHAR(255),
	`photo` BINARY,
	PRIMARY KEY (`id`)
);

CREATE TABLE `SENSORS` (
	`id` INT NOT NULL AUTO_INCREMENT,
	`mac_address` INT NOT NULL UNIQUE,
	PRIMARY KEY (`id`)
);

ALTER TABLE `USER` ADD CONSTRAINT `USER_fk0` FOREIGN KEY
(`company_id`) REFERENCES `COMPANY`(`id`);

ALTER TABLE `INCIDENT` ADD CONSTRAINT `INCIDENT_fk0` FOREIGN KEY
(`mac_address_id`) REFERENCES `SENSORS`(`id`);

ALTER TABLE `INCIDENT` ADD CONSTRAINT `INCIDENT_fk1` FOREIGN KEY
(`location_id`) REFERENCES `LOCATION`(`id`);

ALTER TABLE `SITE` ADD CONSTRAINT `SITE_fk0` FOREIGN KEY
(`company_id`) REFERENCES `COMPANY`(`id`);

ALTER TABLE `LOCATION` ADD CONSTRAINT `LOCATION_fk0` FOREIGN KEY
(`mac_address_id`) REFERENCES `SENSORS`(`id`);

ALTER TABLE `LOCATION` ADD CONSTRAINT `LOCATION_fk1` FOREIGN KEY
(`rackblock_id`) REFERENCES `RACK_BLOCK`(`id`);

ALTER TABLE `RACK_BLOCK` ADD CONSTRAINT `RACK_BLOCK_fk0` FOREIGN KEY
(`site_id`) REFERENCES `SITE`(`id`);
```